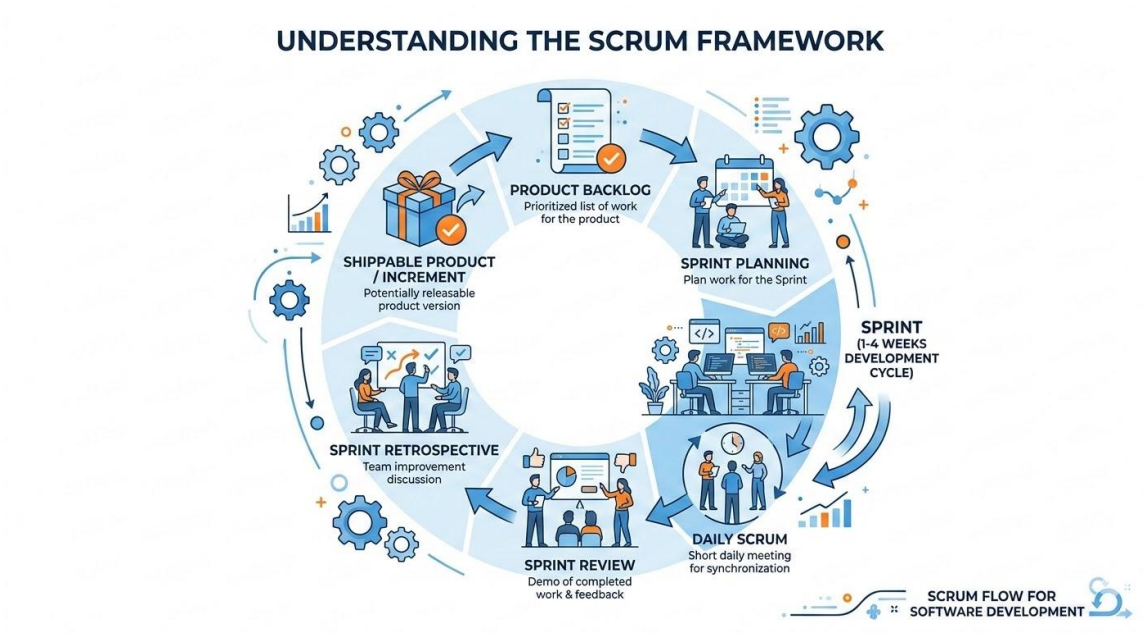
Weekly Progress Report

1. Weekly Progress

During this week, the team reviewed the overall progress of the project. Although we have made noticeable progress in several parts of the platform, we realized that our work organization was not very structured and that the results were not as visible as expected.

For this reason, we decided to adopt the **Scrum methodology**, which is an Agile project management framework that organizes work into short development cycles called **sprints**. Each sprint focuses on a small set of clearly defined tasks, allowing the team to track progress more effectively, improve collaboration between members, and adapt quickly to changes or problems.

Using Scrum will help us structure the development process, assign responsibilities more clearly, and evaluate our progress at the end of each sprint. This approach will also improve team coordination and make it easier to identify development bottlenecks.



During this week, we also **completed the system modeling phase** by creating the necessary UML diagrams that describe the structure and behavior of the platform. These diagrams include:

- Use Case Diagram
- Class Diagram
- Sequence Diagrams
- Activity Diagram

- State Machine Diagram
- Component Diagram
- Deployment Diagram

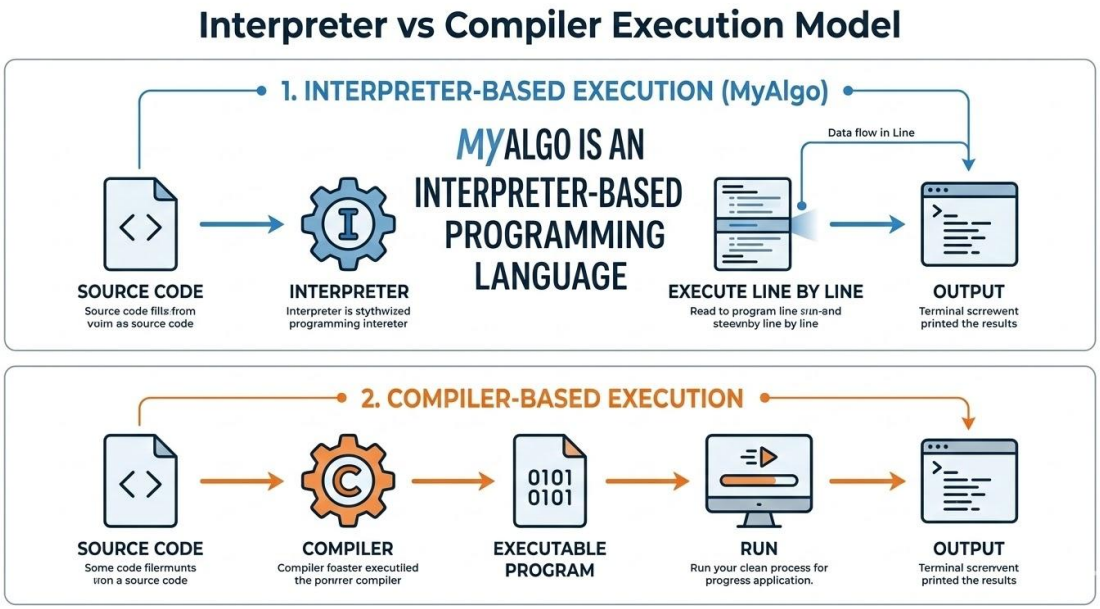
These diagrams help us visualize the architecture of the platform, the interactions between system components, and the behavior of the application in different scenarios.

After completing the first functional version of the platform, we plan to present it to a group of external users in order to collect their feedback and suggestions. This step will help us identify potential improvements and usability issues before continuing the next phases of development.

MyAlgo Language – Creation and Execution Process

How the MyAlgo Language Works

The **MyAlgo language** is an **interpreter-based programming language**, not a compiler-based one.



This means that the program is **not translated into an executable file**. Instead, the source code is **read and executed directly step by step** by the language engine.

When running the command:

```
Windows PowerShell
PS C:\Users\MASSIVE\Desktop\projects\MyAlgo\examples> myalgo hello.algo
Hello, World!
PS C:\Users\MASSIVE\Desktop\projects\MyAlgo\examples>
```

the code goes through several internal stages before producing the final result.

---

1. Reading the Source Code

The first step performed by the program is **reading the .algo file**.

The engine loads the content of the file as plain text:

wirte("Hello World")

This text is then passed to the next stage called **Lexical Analysis**.

---

2. Lexical Analysis (Lexer)

In this stage, the **Lexer** analyzes the text and splits it into smaller elements called **Tokens**.

A **Token** is the smallest meaningful unit in a programming language.

For example, the code:

wirte("Hello World")

is transformed into tokens such as:

WIRTE
(
STRING "Hello World"
)

Each element of the code becomes an independent token.

This process simplifies the structure of the program and makes it easier for the system to understand and process.

---

3. Syntax Analysis (Parser)

After generating the tokens, the next stage is the **Parser**.

The parser takes the tokens and verifies whether the code follows the **grammar rules of the language**.

If the syntax is correct, the parser builds a structure called the **Abstract Syntax Tree (AST)**.

The AST is a **tree representation of the program**.

Example:

```
PrintStatement
    |
    └── "Hello World"
```

This tree represents the **logical meaning of the program**, rather than its textual format.

---

4. AST (Abstract Syntax Tree)

The **Abstract Syntax Tree (AST)** is one of the most important parts of the interpretation process.

It is a **data structure that represents the logic of the program**.

Instead of working directly with the raw text, the engine operates on this **tree structure**.

Example representation:

```
wirte("Hello World")
```

becomes:

```
PrintNode
  |
  └── StringNode("Hello World")
```

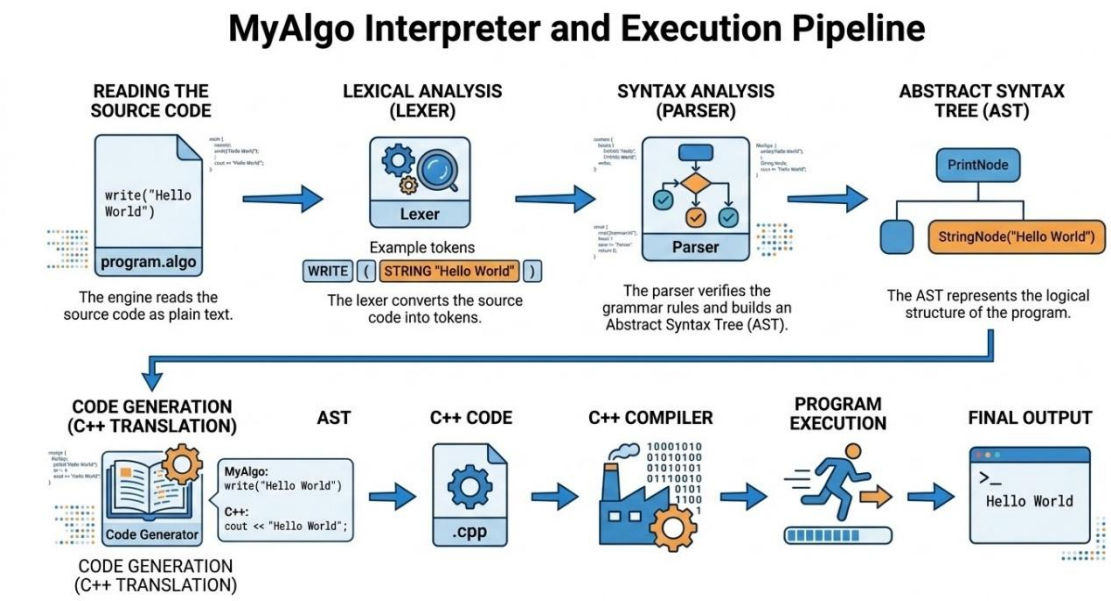
This representation simplifies the execution process and improves the organization of the program logic.

---

5. Interpreter (Execution)

The final stage is the **Interpreter**.

The interpreter **traverses the AST and executes each node**.



This process demonstrates how the language engine interprets and executes the program step by step. The interpreter-based design allows the system to execute students' algorithms instantly and display the results in real time within the platform.

MyAlgo_Platform_Architecture

Conclusion

By running the program:

myalgo hello.algo

we verified that the **MyAlgo language engine works correctly**.

The execution process goes through the following stages:

- Reading the source file
- Converting the code into Tokens (**Lexer**)
- Analyzing the tokens (**Parser**)
- Building the **Abstract Syntax Tree (AST)**
- Executing the AST (**Interpreter**)

These stages represent the **core architecture of interpreter-based programming languages**, enabling the MyAlgo platform to execute algorithms written by students and display results immediately in the learning environment.

MyAlgo_Platform_Architecture